# Short Time Fourier Transform

The short time Fourier transform (STFT) is one of the most basic audio analysis algorithms. It evaluates the spectrogram, in which the time and frequency distribution of the analysed signal can be seen. The STFT is introduced in the following.

[1]

```
import numpy as np
import matplotlib.pyplot as plt
import sys
sys.path.insert(1, '../Basics')
import WaveFileHandling
```

## Problem:

Assuming you have a signal $x(n)$ with increasing frequency over time (a so called sweep). A simple example for a sweep is an electrical engine starting from lowest speed and running up to highest speed.

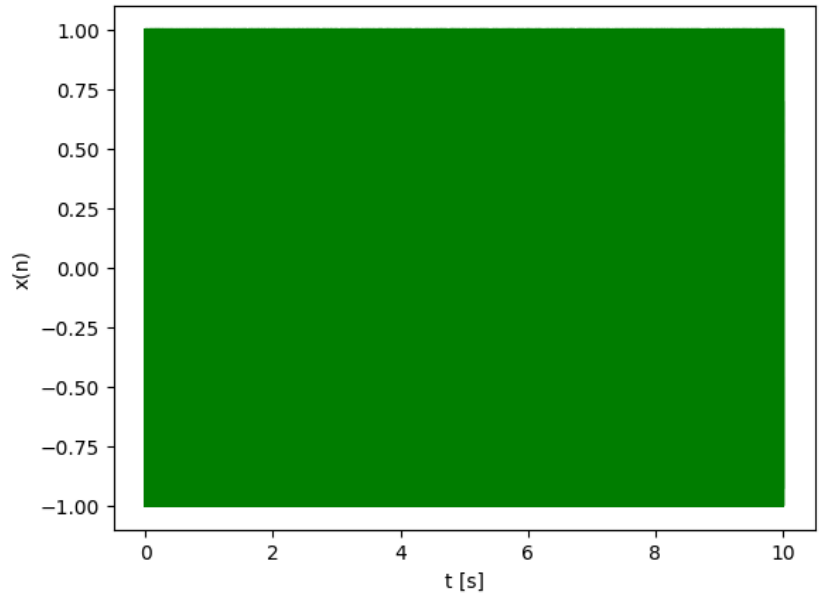
A plot of the corresponding time domain signal in green is hard to analyse due to the high number of samples.

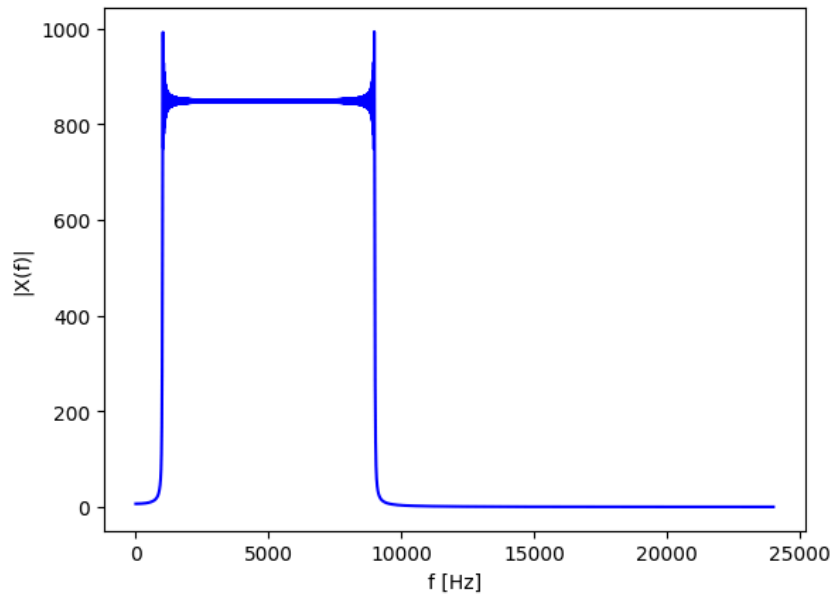
A plot of the magnitude spectrum in blue shows the frequency range of the signal. Unfortunately, you have no information, at which time step, which frequency is active. It is not possible to decide, whether the frequency is increasing or decreasing over time.

Additionally to the plots in time domain and frequency domain, the sweep is stored as a wave file. Listen to it, e.g. with the software audacity, in order to get a feeling, what a sweep is.

[4]

```
r = 48000
t = np.arange(10*r) / r
f = (np.arange(t.shape[0]) / t.shape[0]) * 4000 + 1000
x = np.sin(2*np.pi*f*t)

WaveFileHandling.WriteWaveAsNumpyArray(x, r, 'sweep.wav')

plt.plot(t, x, 'g')
plt.xlabel('t [s]')
plt.ylabel('x(n)')
plt.show()

X = np.fft.fft(x)
NyquistIndex = int(X.shape[0]/2+1)
f_axis = np.arange(NyquistIndex)/X.shape[0]*r
X = np.abs(X[0:NyquistIndex])
plt.plot(f_axis, X, 'b')
plt.xlabel('f [Hz]')
plt.ylabel('|X(f)|')
plt.show()
```




## Solution

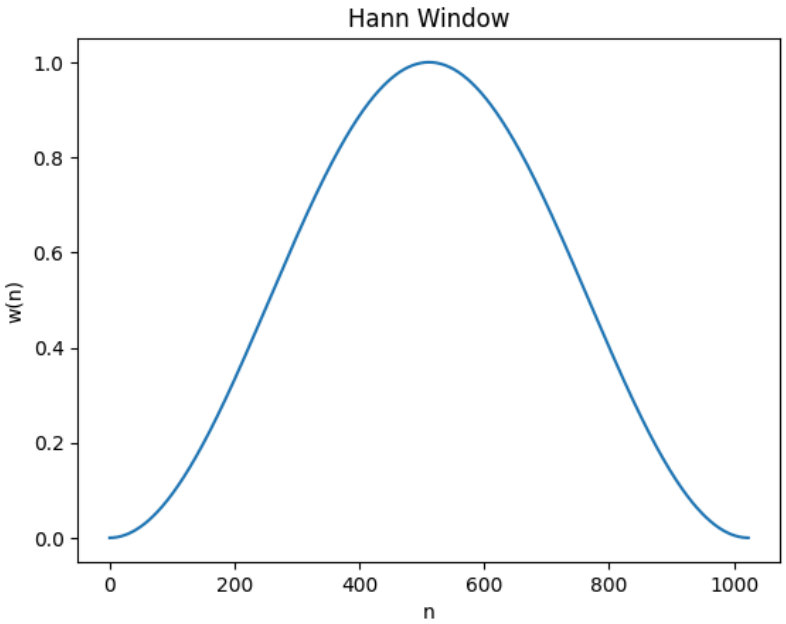
A local analysis of the signal would exhibit the local signal properties (dominant frequencies) and their evolution over time.

The algorithm for this local analysis is the STFT.

As a first step, an analysis window $w(n)$ is defined. In this example, a Hann window is used.

[2]

```
def HannWindow(N):
    return 0.5*(1-np.cos(2*np.pi*(np.arange(N) + 0.5)/N)) # analysis window

ws = 2**10 # window size
hs = int(ws/2) # hop size
w = HannWindow(ws)

plt.plot(w)
plt.xlabel('n')
plt.ylabel('w(n)')
plt.title('Hann Window')
plt.show()
```




The result of the STFT is a so called spectrogram. Its evaluation is explained in the following:

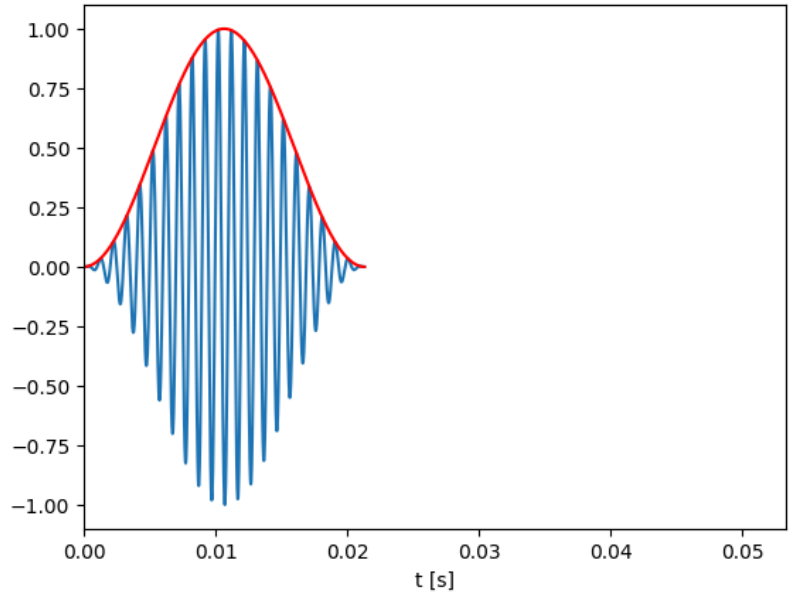
The analysis window is mulitplied element-wise with the first samples of the signal. The product is transformed into a spectrum by the DFT. The resulting spectrum is stored in the first column of the spectrogram.

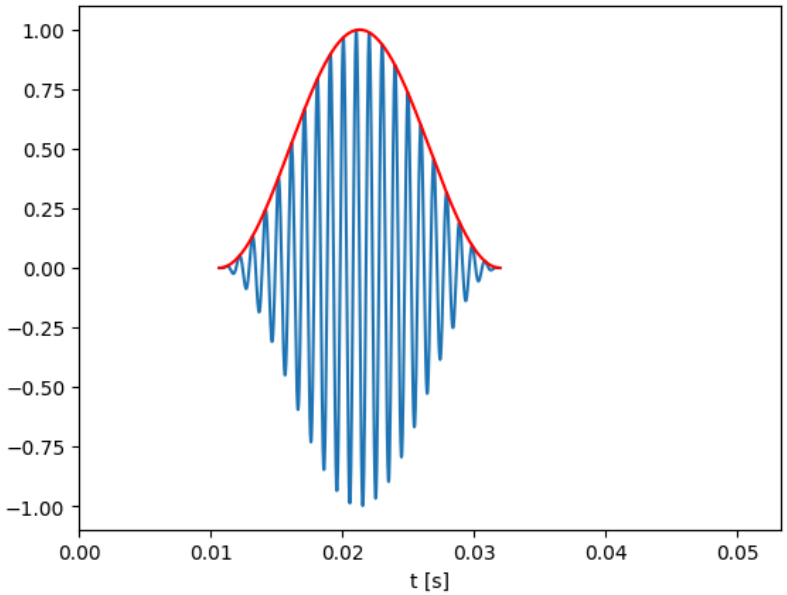
The analysis window is moved by the so called hop size. Again the corresponding region of the signal is multiplied with the analysis window, the DFT is applied and the resulting spectrum is stored in the second column of the spectrogram.

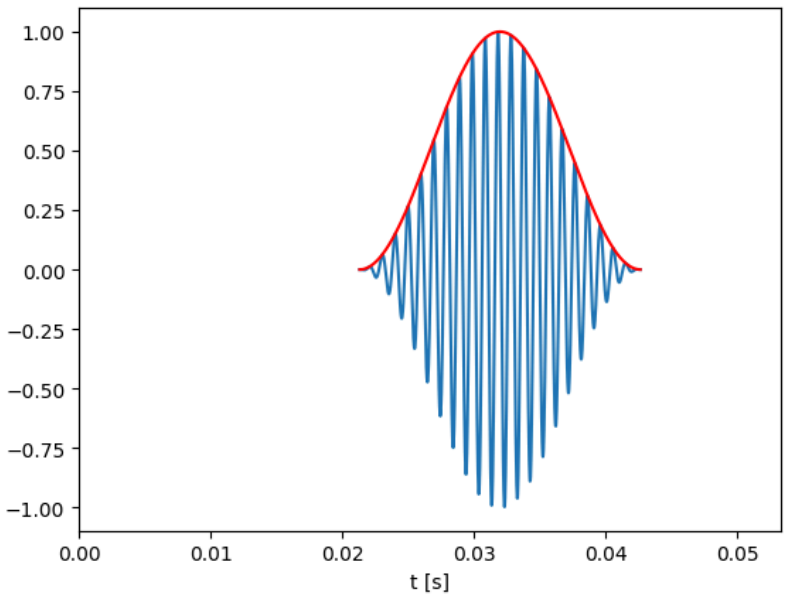
Repeat this algorithm until enough samples of the input signal are analysed.

This algorithm is shown in the following. The first three signal regions are plotted after multipication with the window function.

```
K = 2*ws # use zero padding with a factor of 2
NumberOfColumns = int((x.shape[0] - ws) / hs + 1)
NyquistIndex = K // 2 + 1
X = np.zeros((NyquistIndex, NumberOfColumns), dtype = complex)
for column in range(NumberOfColumns):
    idx1 = column * hs
    idx2 = idx1 + ws
    LocalTimeSignal = x[idx1:idx2] * w
    LocalSpectrum = np.fft.fft(LocalTimeSignal, n = K)
    X[:, column] = LocalSpectrum[0:NyquistIndex]

    # the following code has nothing to do with the STFT and is for visualization
purposes only
    MaxColumnForVisualization = 3
    if column < MaxColumnForVisualization:
        t = np.arange(idx1, idx2) / r
        plt.plot(t, LocalTimeSignal)
        plt.plot(t, w, 'r')
        plt.xlabel('t [s]')
        plt.xlim(0, (MaxColumnForVisualization * hs + ws) / r)
        plt.show()
```





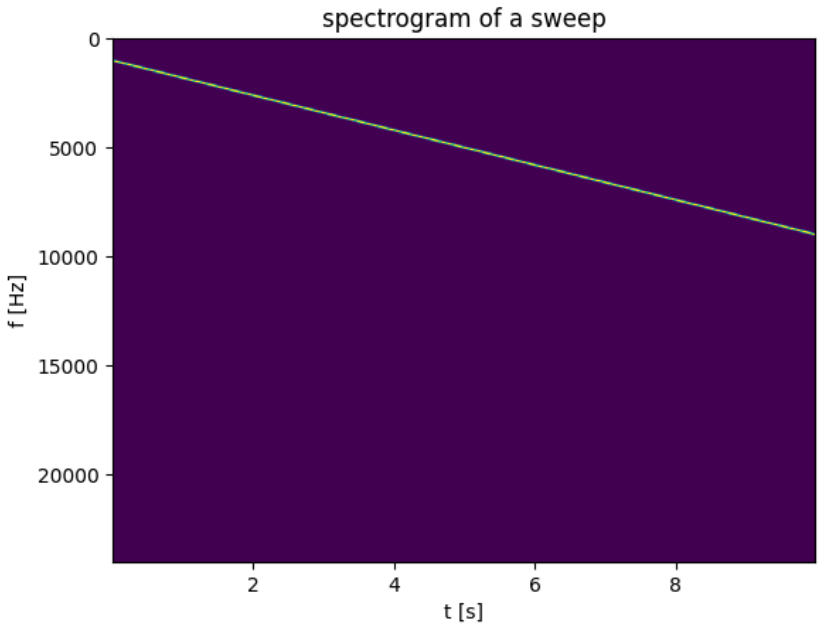



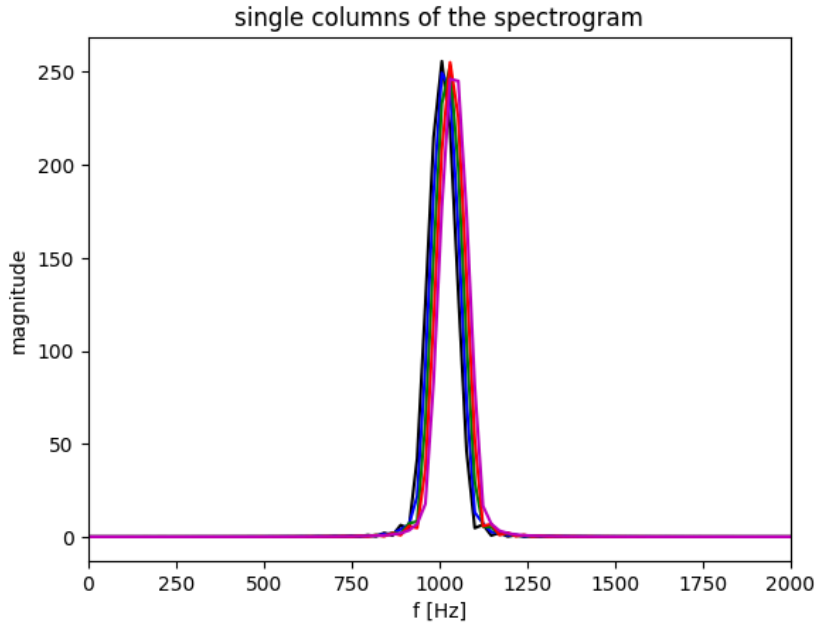

The resulting matrix or the so called spectrogram is plotted in the folowing. The lower magnitudes in the spectra has a dark colour. The higher magnitudes has a lighter colour. By this color-scheme or color-map, the dominant frequencies over the time can be interpreted easily.

The plotted spectrogram in this example shows the increasing frequency over time (note: the frequency axis shows the lowest frequencies in the upper region and the highest frequencies in the lower region).

The second plot shows the first five columns of the spectrogram plotted in different colours in order to show the correspondencies between the plot of the whole spectrogram and the plot of single columns.

```
def PlotSpectrogram(X, t, f, PlotSingleColumns = False):
    plt.imshow(X, interpolation='nearest',aspect='auto', extent =[t.min(), t.max(),
f.max(), f.min()])
    plt.xlabel('t [s]')
    plt.ylabel('f [Hz]')
    plt.title('spectrogram of a sweep')
    plt.show()

    if PlotSingleColumns:
        f_vector = np.arange(X.shape[0])*r/K
        colorvector = 'kbgrm'
        for column in range(len(colorvector)):
            plt.plot(f_vector, X[:, column], colorvector[column])
        plt.xlim(0, 2000)
        plt.xlabel('f [Hz]')
        plt.ylabel('magnitude')
        plt.title('single columns of the spectrogram')
        plt.show()

Delta_f = r / K
Delta_t = hs / r
f = np.arange(X.shape[0]) * Delta_f
t = np.arange(X.shape[1]) * Delta_t + Delta_t/2
PlotSpectrogram(np.abs(X), t, f, PlotSingleColumns = True)
```




## Constant overlap add property

The fade in and fade out property of the analysis window concentrates the focus of the spectral analysis by the DFT on the samples in the mid of the analysis window. For analysing the samples at the end of the analysis window simply the next window is used. For avoiding missing acoustical events by consecutive windowing it is neccessary, that neighbouring windows sum up to a constant.

In the following experiment it is checked, for which combinations of windowsize and hopsize the Hann window has the constant overlap add property necessary for signal analysis and synthesis:

```
def CheckOverlapAdd(w, OverlapInPercent):
    ws = w.shape[0]
    hs = int((1 - OverlapInPercent / 100) * ws)
    NumberOfAnalysisWindows = 10
    WindowSum = np.zeros((NumberOfAnalysisWindows-1)*hs+ws)
    for n in range(NumberOfAnalysisWindows):
        idx1 = n*hs
        idx2 = idx1 + ws
        WindowSum[idx1:idx2] += w
    MiddlePartOfWindowSum = WindowSum[ws:-ws]
    assert np.std(MiddlePartOfWindowSum) < 1e-3, 'no constant window sum beside fade
in and fade out effects'

CheckOverlapAdd(HannWindow(1024), 50)
CheckOverlapAdd(HannWindow(1024), 75)
```

## Time-Frequency Resolution

In the above shown example a signal with continuous signal properties is analysed by the STFT: a sweep.

In the following a signal with two short noisy bursts and two sinusses is assumed.

The two bursts are very close to each other: $10$ milliseconds. Therefore, they can only be separately analysed in a spectrogram with short analysis windows, e.g. $480$ samples for windowsize.

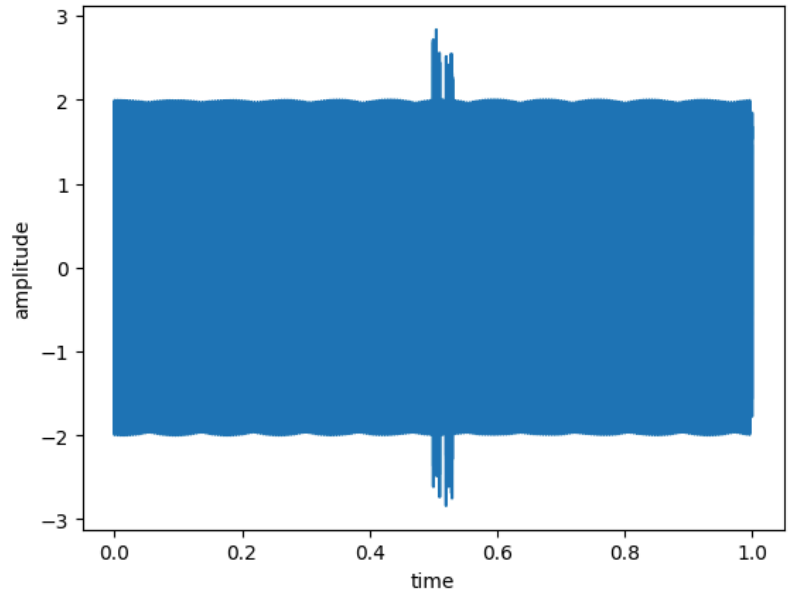
The two sinusses have similar frequencies: $4001$ Hz and $4211$ Hz. They can only be separately analysed in a spectrogram with long analysis windows which results in a good frequency resolution $\Delta_f$, e.g. a windowsize of $9600$ samples.

These both conditions, good time resolution for a windowsize of $480$ samples and good frequency resolution for a windowsize of $9600$ samples cannot be fullfilled at the same time. Therefore, short analysis windows lead to a good time resolution, which allows to separate the two noisy bursts. On the other hand, long analysis windows lead to a good frequency resolution, which allows to separate the two sinusses. In this case, the two bursts are not distinguishable in the spectrogram.

[5]

```
t = np.arange(r)/r
x = np.zeros(t.shape)
# add two sinusses with the given frequencies
for f in [4001, 4211]:
```

```
    phi = f**2 # arbitrary phase
    x += np.sin(2*np.pi*f*t + phi)

# add two short noisy bursts
BurstLengthInMilliseconds = 10
BurstLengthInSamples = int(BurstLengthInMilliseconds * r / 1000)
for n in range(2):
    idx1 = int(t.shape[0]/2 + 2*n*BurstLengthInSamples)
    idx2 = idx1 + BurstLengthInSamples
    x[idx1:idx2] += np.random.rand(BurstLengthInSamples)*2-1

plt.plot(t, x)
plt.xlabel('time')
plt.ylabel('amplitude')
plt.show()

def Spectrogram(x, WindowSize, FFTLen = None, HopSize = None):
    if HopSize is None:
        HopSize = WindowSize // 2 # 50 % of overlap
    if FFTLen is None:
        FFTLen = WindowSize
    NumberOfColumns = int(np.floor((x.shape[0] - WindowSize) / HopSize) + 1)
    X = np.zeros((FFTLen // 2 + 1, NumberOfColumns), dtype = complex)
    w = HannWindow(WindowSize)
    for column in range(NumberOfColumns):
        idx1 = column * HopSize
        idx2 = idx1 + WindowSize
        x_Block = x[idx1:idx2] * w
        X[:, column] = np.fft.rfft(x_Block, n = FFTLen)
    return X

K = 9600
hs = K // 2
X_9600 = Spectrogram(x, K, K, hs)
Delta_f = r / K
Delta_t = hs / r
f = np.arange(X_9600.shape[0]) * Delta_f
t = np.arange(X_9600.shape[1]) * Delta_t + Delta_t/2
PlotSpectrogram(20*np.log10(np.abs(X_9600)), t, f)

K = 480
hs = K // 2
X_480 = Spectrogram(x, K, K, hs)
Delta_f = r / K
Delta_t = hs / r
f = np.arange(X_480.shape[0]) * Delta_f
t = np.arange(X_480.shape[1]) * Delta_t + Delta_t/2
PlotSpectrogram(20*np.log10(np.abs(X_480)), t, f)
```




```
---------------------------------------------------------------------------
NameError                                 Traceback (most recent call last)
Cell In[5], line 43
     41 f = np.arange(X_9600.shape[0]) * Delta_f
     42 t = np.arange(X_9600.shape[1]) * Delta_t + Delta_t/2
---> 43 PlotSpectrogram(20*np.log10(np.abs(X_9600)), t, f)
     45 K = 480
     46 hs = K // 2

NameError: name 'PlotSpectrogram' is not defined
```

## Time resolution of the human ear

The hearing cells in the human ear can fire roughly once per millisecond. By this, the hopsize of the human ear can be approximated by this value. For normal audio analysis by a software, this hopsize of the human ear is far to precise. A good rule of thumb are hopsizes for roughly 10 milliseconds for speech analysis and roughly 100 for music analysis.

## STFT as preprocessing

For audio analysis, classification, transcription and other tasks, the STFT is a typical input feature for Neural Networks. Other standard preprocessing algorithms usually starts with the STFT:

### Mel-Spectrogram

For a mel-spectrogram the frequency axis of the spectrogram is compressed logarithmically. By this step, the linear frequency axis is transformed into a logarithmic frequency axis, e.g. the mel-axis, see also Bark-scale for more informations regarding the logarithmic frequency resolution of human ear. A mel-spectrogram uses typically $24$ logarithmically spaced bandpass filters for compressing the spectrogram. By this, speech intelligibility should still be possible, as outlined in Bark-scale. Additionally, by this data compression from several hundreds linear frequency bins to $24$ logarithmically spaced bandpasses the amount of input data for the classification stage is reduced, which reduces the amount of necessary training data for successful training of a classification algorithm.

### Log-Mel-Spectrogram

Beside the logarithmic frequency resolution, humans receive loudness of sound effects in a logarithmic way, see also Decibel and SNR. Therefore, the magnitudes of a mel-spectrogram are often compressed logarithmically, such forming the log-mel-spectrogram.

### Mel-Frequency-Cepstral-Coefficients

The Mel-Frequency-Cepstral-Coefficients (MFCC) are standard audio-features for classification for smaller Neural Networks. With growing networks and more efficient algorithms and better hardware, the Log-Mel-Spectrogram is more often used as preprocessing for audio signals.

MFCC are evaluated by appylying a discrete cosine transform (DCT) on the frequency axis of a log-mel-spectrogram followed by dropping the lowest frequency and a set of the highest frequencies.

The MFCC are strongly related to the source-filter-model of speech production.

Because of our later usage of PyTorch for classification, the implementation of these preprocessing steps in PyTorch are shown in the following.

```
###
https://docs.pytorch.org/audio/main/generated/torchaudio.transforms.MelSpectrogram.h
tml
import torchaudio

waveform, sample_rate = torchaudio.load("../Audio/P501_D_EN_fm_SWB_48k.wav",
normalize=True)

### sample code for evaluating the spectrogram with pytorch
transform = torchaudio.transforms.Spectrogram(n_fft = 1024, hop_length = 512)
spectrogram = transform(waveform)

### sample code for evaluating the mel-spectrogram with pytorch
transform = torchaudio.transforms.MelSpectrogram(sample_rate, n_mels = 40, n_fft =
1024, hop_length = 512)
mel_spectrogram = transform(waveform)

### sample code for evaluating the MFCC with pytorch
transform = torchaudio.transforms.MFCC(sample_rate, n_mfcc=12, melkwargs={"n_fft":
512, "hop_length": 160, "n_mels": 24, "center": False})
mfcc = transform(waveform)
```

## Programming exercise

The inverse short time Fourier transform works with the overlap add algorithm:

Each column of the spectrogram is transformed into time domain by the inverse DFT. This local time domain signal $y_{\text{block}}$ is moved to the position corresponding to this column of the spectrogram. At this position, $y_{\text{block}}$ is added up into the output signal $y$.

If the window functions used for analyses add up to the constant $1$, there is no error between input signal $x$ and output signal $y$. Therefore, it is important to know, at which combinations of windowsize and hopsize the analysis windows add up to a constant.

The length of the FFT (FFTLen) should be a power of $2$ at least if you are evaluating the FFT on a low cost processor. Implement the procedure NextPowerOf2, which finds an integer value, which is a power of two and which is greater or equal than WindowSize.

Implement the procedure ISTFT.

[6]

```
def NextPowerOf2(WindowSize):
    FFTLen = 0
    ### solution
    # Naechste Zweierpotenz finden, die groesser/gleich WindowSize ist.
    FFTLen = 1
    while FFTLen < WindowSize:
        FFTLen *= 2
    ### solution
    return FFTLen

def GetWindowSize(X, WindowSize):
    ### solution
    # Falls nicht gegeben: aus der Anzahl der Frequenzbins auf FFTLen
rueckschliessen.
    if WindowSize is None:
        WindowSize = (X.shape[0] - 1) * 2
    ### solution
    return WindowSize

def GetHopSize(WindowSize, HopSize):
    ### solution
    # Standardannahme: 50 % Ueberlappung.
    if HopSize is None:
        HopSize = WindowSize // 2
    ### solution
    return HopSize

def GetOutputLength(X, WindowSize, HopSize):
    OutputLength = 0
    ### solution
    # Bei M Spalten gilt: letzte Blockposition + Fensterlaenge.
```

```
    NumberOfColumns = X.shape[1]
    OutputLength = (NumberOfColumns - 1) * HopSize + WindowSize
    #### solution
    return OutputLength


def ISTFT(X, WindowSize = None, HopSize = None):
    y = 0
    WindowSize = GetWindowSize(X, WindowSize)
    HopSize = GetHopSize(WindowSize, HopSize)
    y = np.zeros((GetOutputLength(X, WindowSize, HopSize)))
    ### solution
    # Weighted overlap-add: mit Hann-Fenster synthetisieren und lokal normieren.
    FFTLen = (X.shape[0] - 1) * 2
    w = HannWindow(WindowSize)
    Normalization = np.zeros_like(y)

    for column in range(X.shape[1]):
        idx1 = column * HopSize
        idx2 = idx1 + WindowSize
        y_block = np.fft.irfft(X[:, column], n=FFTLen)[:WindowSize]
        y[idx1:idx2] += y_block * w
        Normalization[idx1:idx2] += w**2

    Valid = Normalization > 1e-12
    y[Valid] /= Normalization[Valid]
    ### solution
    return y

import unittest

class TestProgrammingExercise(unittest.TestCase):

    def test_PowerOf2_Log2IsInteger(self):
        x = np.random.randint(100) + 1
        y = NextPowerOf2(x)
        z = np.log2(y)
        self.assertAlmostEqual(int(z), z, 1e-3)

    def test_PowerOf2_1(self):
        x = np.random.randint(100) + 1
        y = NextPowerOf2(x)
        self.assertGreaterEqual(y, x)

    def test_PowerOf2_2(self):
        x = np.random.randint(100) + 1
        y = NextPowerOf2(x)
        self.assertGreaterEqual(x, y/2)

    def test_WindowSizeNone(self):
        X = np.random.randn(47, 100)
        ws = GetWindowSize(X, None)
        self.assertEqual(ws, 92)
        X = np.random.randn(53, 100)
        ws = GetWindowSize(X, None)
        self.assertEqual(ws, 104)

    def test_WindowSizeDefined(self):
        X = np.random.randn(47, 100)
        WindowSize = np.random.randint(500)
        self.assertEqual(GetWindowSize(X, WindowSize), WindowSize)

    def test_HopSizeNone(self):
        hs = GetHopSize(512, None)
        self.assertEqual(hs, 256)
        hs = GetHopSize(1000, None)
        self.assertEqual(hs, 500)

    def test_HopSizeDefined(self):
        HopSize = np.random.randint(500)
        self.assertEqual(GetHopSize(np.random.randint(1000), HopSize), HopSize)

    def test_OutputLength(self):
        ws = 1000
        hs = ws // 2
        FFTLen = ws
        x = np.random.randn(50000)
        X = Spectrogram(x, ws, FFTLen, hs)
        y = ISTFT(X)
        self.assertEqual(x.shape[0], y.shape[0])

    def test_ISTFT(self):
        ws = 1000
        hs = ws // 2
        FFTLen = ws
        x = np.random.randn(50000)
        X = Spectrogram(x, ws, FFTLen, hs)
        y = ISTFT(X)
        x = x[:y.shape[0]]
        Offset = ws - hs
        x = x[Offset:-Offset]
        y = y[Offset:-Offset]
        self.assertGreater(10*np.log10(np.sum(x**2) / np.sum((x-y)**2)), 200)

    def test_ISTFT_hs(self):
        hs = 500
        ws = hs * (np.random.randint(4) + 2)
        FFTLen = ws
        x = np.random.randn(50000)
        X = Spectrogram(x, ws, FFTLen, hs)
        y = ISTFT(X, ws, hs)
        x = x[:y.shape[0]]
        Offset = ws - hs
        x = x[Offset:-Offset]
        y = y[Offset:-Offset]
        a = np.sum(x*y)/np.sum(y**2)
        self.assertGreater(10*np.log10(np.sum(x**2) / np.sum((x-a*y)**2)), 200)

    def test_ISTFT_FFTLen(self):
        hs = 500
        ws = hs * (np.random.randint(4) + 2)
        FFTLen = NextPowerOf2(ws + np.random.randint(ws))
```

```
        x = np.random.randn(50000)
        X = Spectrogram(x, ws, FFTLen, hs)
        y = ISTFT(X, ws, hs)
        x = x[:y.shape[0]]
        Offset = ws - hs
        x = x[Offset:-Offset]
        y = y[Offset:-Offset]
        a = np.sum(x*y)/np.sum(y**2)
        self.assertGreater(10*np.log10(np.sum(x**2) / np.sum((x-a*y)**2)), 200)

    def test_ISTFT_CorrectScaling(self):
        hs = 500
        ws = hs * (np.random.randint(4) + 2)
        FFTLen = NextPowerOf2(ws + np.random.randint(ws))
        x = np.random.randn(50000)
        X = Spectrogram(x, ws, FFTLen, hs)
        y = ISTFT(X, ws, hs)
        x = x[:y.shape[0]]
        Offset = ws - hs
        x = x[Offset:-Offset]
        y = y[Offset:-Offset]
        self.assertGreater(10*np.log10(np.sum(x**2) / np.sum((x-y)**2)), 200)

unittest.main(argv=['''], verbosity=2, exit=False)
```

```
Collapse
test_HopSizeDefined (__main__.TestProgrammingExercise.test_HopSizeDefined) ... ok
test_HopSizeNone (__main__.TestProgrammingExercise.test_HopSizeNone) ... ok
test_ISTFT (__main__.TestProgrammingExercise.test_ISTFT) ... ok
test_ISTFT_CorrectScaling
(__main__.TestProgrammingExercise.test_ISTFT_CorrectScaling) ... ok
test_ISTFT_FFTLen (__main__.TestProgrammingExercise.test_ISTFT_FFTLen) ... ok
test_ISTFT_hs (__main__.TestProgrammingExercise.test_ISTFT_hs) ... ok
test_OutputLength (__main__.TestProgrammingExercise.test_OutputLength) ... ok
test_PowerOf2_1 (__main__.TestProgrammingExercise.test_PowerOf2_1) ... ok
test_PowerOf2_2 (__main__.TestProgrammingExercise.test_PowerOf2_2) ... ok
test_PowerOf2_Log2IsInteger
(__main__.TestProgrammingExercise.test_PowerOf2_Log2IsInteger) ... ok
test_WindowSizeDefined (__main__.TestProgrammingExercise.test_WindowSizeDefined) ...
ok
test_WindowSizeNone (__main__.TestProgrammingExercise.test_WindowSizeNone) ... ok

----------------------------------------------------------------------
Ran 12 tests in 0.073s

OK
```

```
<unittest.main.TestProgram at 0x2055d86d2b0>
```

# Exam preparation

An audio signal of length $3$ seconds has a sampling rate of $r = 16$ kHz. The frequency resolution $\Delta f$ should be better than $10$ Hz. The time resolution should be better than $50$ spectra per second. The overlap of two neighbouring windows is $50$ %. Evaluate the minimum values of $K$ and $N$. Is zero padding necessary? Evaluate the memory consumption of this spectrogram, if each real valued parameter needs 4 Bytes and each complex valued parameter needs 8 Bytes of memory.

According to the above plotted spectrograms, which statement is true: The smaller the windowsize, the better the time resolution of a spectrogram. The greater the windowsize, the better the time resolution of a spectrogram.

According to the above plotted spectrograms, which statement is true: The smaller the windowsize, the better the frequency resolution of a spectrogram. The greater the windowsize, the better the frequency resolution of a spectrogram.

An audio signal of length $5$ seconds is analysed by a STFT. The windowsize / blocksize is $N = 100$ ms. The overlap is $75$ %. How many blocks are analysed?

An audio signal of length $5$ seconds is analysed by a STFT. The hopsize corresponds to the time resolution of the human ear. The spectra are transformed to the frequency resolution of the human ear. The frequency range, stored in the spectrogram is $0 \leq b \leq 24$ Bark. How many values are stored in the spectrogram?

A window function is defined by $w(n) = \left(0.5 - 0.5 \cdot \cos\left(2\pi \frac{n+0.5}{N}\right)\right)^{\alpha}$ with $0 \leq n < N$ and $0 \leq \alpha$. Which hopsize leads to a constant overlap add independent from the parameter $\alpha$? For which $\alpha$ $w(n)$ is euqal to the Hann window?

# Exam preparation - verständliche Musterlösung

## 1) Mindestwerte für $K$ und $N$, Zero Padding, Speicher

Gegeben: Signallänge $T = 3\,\text{s}$, Abtastrate $r = 16\,\text{kHz}$, Überlappung $50\%$.

Frequenzauflösung:

$$\Delta f = \frac{r}{K} < 10\,\text{Hz} \Rightarrow K > \frac{16000}{10} = 1600$$

Damit ist der kleinste ganzzahlige Wert $K_{\min} = 1601$ (praktisch FFT-freundlich meist $K = 2048$).

Zeitauflösung: "besser als 50 Spektren/s" bedeutet

$$\frac{1}{\Delta t} > 50 \Rightarrow \Delta t < 20\,\text{ms}$$

Mit $50\%$ Überlappung gilt $H = \frac{N}{2}$ und $\Delta t = \frac{H}{r}$:

$$\frac{N}{2r} < 0.02 \Rightarrow N < 640$$

Grenzfall (genau 50 Spektren/s): $N = 640$, strikt "besser" also z. B. $N = 638$.

Wichtige Schlussfolgerung:

- Für die Frequenzauflösung braucht man $K \gtrsim 1600$.
- Für die Zeitauflösung muss $N < 640$ sein.
- Also: $K > N$ notwendig, daher ist Zero Padding notwendig.

Beispiel mit $N = 640$, $H = 320$, $K = 1600$:

- Signalpunkte: $L = 3 \cdot 16000 = 48000$
- Anzahl Zeitspalten:

$$M = \left\lfloor \frac{L - N}{H} \right\rfloor + 1 = \left\lfloor \frac{48000 - 640}{320} \right\rfloor + 1 = 149$$

- Frequenzzeilen (rFFT): $K/2 + 1 = 801$
- Komplexe Spektrogrammwerte: $801 \cdot 149 = 119349$
- Speicher (8 Byte pro komplexem Wert):

$$119349 \cdot 8 = 954792\,\text{Byte} \approx 0.91\,\text{MiB}$$

(Häufige Praxis: $K = 2048$ statt $1600/1601$.)

### 2) Zeitauflösung

Richtig ist:

- Je kleiner die Fenstergröße, desto besser die Zeitauflösung.

### 3) Frequenzauflösung

Richtig ist:

- Je größer die Fenstergröße, desto besser die Frequenzauflösung.

### 4) Anzahl Blöcke bei $5\,\text{s}$, $N = 100\,\text{ms}$, Überlappung $75\%$

Hopsize:

$$H = (1 - 0.75) \cdot 100\,\text{ms} = 25\,\text{ms}$$

Anzahl Blöcke:

$$M = \left\lfloor \frac{5000 - 100}{25} \right\rfloor + 1 = 196 + 1 = 197$$

Also werden 197 Blöcke analysiert.

### 5) Anzahl gespeicherter Werte bei Bark-Darstellung

Hopsize wie menschliches Ohr: etwa $1\,\text{ms}$. Bei $5\,\text{s}$ ergibt das ca. $5000$ Zeitpositionen.

Für den Bark-Bereich $0 \le b \le 24$ verwendet man in der Praxis meist 24 Bänder. Dann:

$$5000 \cdot 24 = 120000$$

Also etwa 120000 gespeicherte Werte.

(Hinweis: Wenn man die Endpunkte als 25 diskrete Stufen zählt, wären es $5000 \cdot 25 = 125000$.)

### 6) Fenster $w(n) = \left(0.5 - 0.5\cos\left(2\pi \frac{n+0.5}{N}\right)\right)^{\alpha}$

- Für das klassische Hann-Fenster gilt $\alpha = 1$.
- Der übliche COLA-Fall beim Hann-Fenster ist $H = N/2$ (50 % Überlappung).
- Bei allgemeinem $\alpha \neq 1$ ist die exakte COLA-Bedingung im Allgemeinen $\alpha$-abhängig; der einfache $H = N/2$-Merksatz gilt exakt für $\alpha = 1$.

## Summary

After working with this Jupyter Notebook you should be able to explain the following topics:

- What is the motivation for using a STFT for analysis?
- What is the advantage of a constant overlap add scenario?
- What is the relationship between windowsize, hopsize and used window?
- What is time resolution and frequency resolution?

## Summary - Antworten

**Motivation für die STFT:** Ein normales FFT-Spektrum zeigt nur, *welche* Frequenzen insgesamt vorkommen, aber nicht *wann* sie auftreten. Die STFT liefert eine Zeit-Frequenz-Darstellung (Spektrogramm) und macht damit zeitliche Änderungen wie Sweeps, kurze Bursts oder Übergänge sichtbar.

**Vorteil von Constant Overlap Add (COLA):** Wenn sich benachbarte Fenster beim Überlappen zu einer konstanten Summe addieren, kann das Signal durch Overlap-Add ohne Amplitudenfehler rekonstruiert werden. Das ist wichtig für eine saubere ISTFT (Analyse und Synthese passen zusammen).

**Zusammenhang zwischen WindowSize, HopSize und Fensterfunktion:** `WindowSize` bestimmt die Blocklänge, `HopSize` den Abstand zwischen zwei Blöcken (damit die Überlappung), und die Fensterfunktion (z. B. Hann) formt die Blockränder, um Spektralleckage zu reduzieren. Diese drei Größen müssen zusammenpassen; z. B. funktioniert Hann typischerweise gut mit 50 % Überlappung ( `HopSize = WindowSize / 2` ) für COLA.

**Zeitauflösung und Frequenzauflösung:** Zeitauflösung beschreibt, wie genau Änderungen über die Zeit lokalisiert werden können (abhängig vor allem von `HopSize` / `WindowSize` ). Frequenzauflösung

beschreibt, wie fein benachbarte Frequenzen getrennt werden können (abhängig vor allem von `WindowSize` bzw. FFT-Länge). Grundkonflikt: kurze Fenster -> gute Zeitauflösung, lange Fenster -> gute Frequenzauflösung.